EXP5

1 Logistic Regression Program

1.1 Import the required libraries

[4]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score
```

1.2 Import the dataset

[5]:
```
diabetes = pd.read_csv('diabetes.csv')
diabetes.columns
```

[5]:
```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

[6]:
```
diabetes.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

1.3 Remove the outcome feature to train the model

make y-axis the outcome

[7]:
```
X = diabetes.drop('Outcome', axis=1) # All columns except 'Outcome' are features
y = diabetes['Outcome'] # 'Outcome' is the target variable
feature_names = X.columns # Get feature names from the X DataFrame
print(feature_names)
```

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='str')
```

[8]:
```
print(X.shape)
print(y.shape)
```

```
(768, 8)
(768,)
```

1.4 Scale the features

[9]:
```
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)

X_scaled = (X - mean) / std

print("Mean of each feature:")
print(mean)

print("\nStandard Deviation of each feature:")
print(std)
```

```
Mean of each feature:
Pregnancies                 3.845052
Glucose                   120.894531
BloodPressure              69.105469
SkinThickness              20.536458
Insulin                    79.799479
BMI                        31.992578
DiabetesPedigreeFunction    0.471876
Age                        33.240885
dtype: float64

Standard Deviation of each feature:
Pregnancies                 3.367384
Glucose                    31.951796
BloodPressure              19.343202
SkinThickness              15.941829
Insulin                   115.168949
BMI                         7.879026
```

```
DiabetesPedigreeFunction      0.331113
Age                          11.752573
dtype: float64
```

1.5 Perform the train test split for both scaled and non scaled values

[10]:
```
# First, perform train-test split on the original, non-scaled data
X_train_non_scaled, X_test_non_scaled, y_train_non_scaled, y_test_non_scaled =␣
 ↪train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
# Second, preform train-test split on scaled data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)
```

1.6 Create the sigmoid function and do the scaled model training

[11]:
```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Initialize weights and bias for scaled regession
num_features = X_train.shape[1]
weights = np.zeros(num_features)
bias = 0

# Hyperparameters for scaled model
learning_rate = 0.0001
num_iterations = 5000

# Gradient Descent for scaled model
for _ in range(num_iterations):
    # Calculate linear combination (z)
    z = np.dot(X_train, weights) + bias
    # Calculate predicted probabilities
    y_pred_proba = sigmoid(z)
    # Calculate gradients
    dw = (1/len(X_train)) * np.dot(X_train.T, (y_pred_proba - y_train))
    db = (1/len(X_train)) * np.sum(y_pred_proba - y_train)
    # Update weights and bias
    weights -= learning_rate * dw
```

```
    bias -= learning_rate * db

# Make predictions on the test set
z_test = np.dot(X_test, weights) + bias
y_pred_proba_manual = sigmoid(z_test)
y_pred_manual = (y_pred_proba_manual >= 0.5).astype(int)
```

1.7 Create a stable sigmoid function to return negative stable values and train the non scaled model

[12]:
```
from sklearn.metrics import accuracy_score, precision_score, f1_score,␣
↪recall_score

def stable_sigmoid(z):
    # Numerically stable sigmoid function
    return np.where(z >= 0,
                    1 / (1 + np.exp(-z)),
                    np.exp(z) / (1 + np.exp(z)))



# Initialize weights and bias for the non-scaled model
num_features_non_scaled = X_train_non_scaled.shape[1]
weights_non_scaled = np.zeros(num_features_non_scaled)
bias_non_scaled = 0

# Hyperparameters (Using smaller learning rate for non-scaled stability)
learning_rate_non_scaled = 0.0001
num_iterations_non_scaled = 5000

# Gradient Descent for non-scaled data
for _ in range(num_iterations_non_scaled):
    z_non_scaled = np.dot(X_train_non_scaled, weights_non_scaled) +␣
↪bias_non_scaled
    y_pred_proba_non_scaled = stable_sigmoid(z_non_scaled)
    dw_non_scaled = (1/len(X_train_non_scaled)) * np.dot(X_train_non_scaled.T,␣
↪(y_pred_proba_non_scaled - y_train_non_scaled))
    db_non_scaled = (1/len(X_train_non_scaled)) * np.
↪sum(y_pred_proba_non_scaled - y_train_non_scaled)
    weights_non_scaled -= learning_rate_non_scaled * dw_non_scaled
    bias_non_scaled -= learning_rate_non_scaled * db_non_scaled

# Make predictions on the test set for non-scaled data
z_test_non_scaled = np.dot(X_test_non_scaled, weights_non_scaled) +␣
↪bias_non_scaled
y_pred_proba_manual_non_scaled = stable_sigmoid(z_test_non_scaled)
y_pred_manual_non_scaled = (y_pred_proba_manual_non_scaled >= 0.5).astype(int)
```

```
mse_lr_non_scaled = mean_squared_error(y_test_non_scaled,␣
 ↪y_pred_manual_non_scaled)
r2_lr_non_scaled = r2_score(y_test_non_scaled, y_pred_manual_non_scaled)
accuracy_lr_non_scaled = accuracy_score(y_test_non_scaled,␣
 ↪y_pred_manual_non_scaled)
precision_lr_non_scaled = precision_score(y_test_non_scaled,␣
 ↪y_pred_manual_non_scaled)
f1_lr_non_scaled = f1_score(y_test_non_scaled, y_pred_manual_non_scaled)
recall_lr_non_scaled = recall_score(y_test_non_scaled, y_pred_manual_non_scaled)

# Calculate additional metrics for the scaled model
accuracy_lr = accuracy_score(y_test, y_pred_manual)
precision_lr = precision_score(y_test, y_pred_manual)
f1_lr = f1_score(y_test, y_pred_manual)
recall_lr = recall_score(y_test, y_pred_manual)

# Combine results into a DataFrame
results = pd.DataFrame({
    'Model': ['Logistic Regression (Scaled Features)', 'Logistic Regression␣
 ↪(Non-Scaled Features)']  ,
    'Accuracy': [accuracy_lr, accuracy_lr_non_scaled],
    'Precision': [precision_lr, precision_lr_non_scaled],
    'F1 Score': [f1_lr, f1_lr_non_scaled],
    'Recall': [recall_lr, recall_lr_non_scaled]
})

print("\nPerformance Comparison")
print(results)
```

```
Performance Comparison
                                    Model  Accuracy  Precision  F1 Score  \
0      Logistic Regression (Scaled Features)  0.707792   0.583333  0.608696
1  Logistic Regression (Non-Scaled Features)  0.707792   0.666667  0.470588

     Recall
0  0.636364
1  0.363636
```